

Technical Architecture and System Documentation

Project: Zero-Knowledge Proof (ZKP) Credential Verifier
Team: Avengers
Version: 1.0.0
Date: October 2, 2026

Repository: https://github.com/Balavardhan01/ZKP-Credential-Verifier_Avengers_Avirbhav
Live Demo: [Insert Live URL Here]

1. Executive Summary

The ZKP Credential Verifier is an enterprise-grade identity protocol engineered to facilitate trustless, privacy-preserving compliance checks. The system resolves the inherent conflict between corporate auditing requirements and end-user privacy by enabling individuals to cryptographically prove they meet specific policy constraints (e.g., GPA thresholds, degree classifications, graduation years) without exposing underlying sensitive data.

By leveraging Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge (zk-SNARKs) computed entirely on the client side, the protocol ensures that Personally Identifiable Information (PII) is never transmitted across the network or stored in relying party databases.

2. Core System Features

- **Localized zk-SNARK Execution:** Groth16 circuit evaluation is performed strictly within the user's local browser enclave, ensuring absolute data sovereignty.
 - **Visual Selective Disclosure:** The user interface implements dynamic data obfuscation to visually indicate the shielding of PII during proof generation.
 - **Account Abstraction (ERC-4337):** The architecture utilizes a simulated Paymaster integration to subsidize network gas fees, abstracting blockchain friction away from the end-user.
 - **Dynamic Policy Engine:** Relying parties (Verifiers) possess the capability to dynamically configure logical assertion thresholds on demand.
 - **Live SIEM & Immutable Ledger:** The system integrates a real-time cryptographic pairing matrix and records all validation events to an immutable on-chain audit log.
 - **Verifiable Presentation Export:** Supports the generation and export of W3C-standard JSON-LD cryptographic presentations for enterprise archiving and compliance auditing.
-

3. System Architecture

The platform is segregated into three isolated environments to reflect real-world cryptographic infrastructure and enforce strict separation of concerns:

3.1. Issuer Portal (Institution)

A centralized registrar interface responsible for minting secure, digitally signed identity payloads. Institutions define standard user attributes (Name, College, Degree, Year Passed, Cumulative GPA) and generate a cryptographic signature to authenticate the dataset.

3.2. Identity Vault (Prover Client)

A secure, client-side wallet interface where end-users maintain custody of their identity shards. This environment operates as the cryptographic prover, executing Groth16 circuit constraints locally to generate zero-knowledge proofs.

3.3. Enterprise Verifier (Relying Party)

A compliance dashboard simulating a corporate HR or security infrastructure. Verifiers define logical policies, intercept incoming proofs, execute elliptic curve pairing checks, and record validation outcomes to an immutable ledger.

3.4. Technical Stack

- **Frontend Ecosystem:** Next.js (App Router), React, TypeScript, Tailwind CSS.
 - **Cryptography & Smart Contracts:** Solidity, Hardhat, Ethers.js v6, Groth16 SNARK simulations (BN128 curve).
 - **State Synchronization:** React Context API coupled with BroadcastChannel API for real-time, multi-client state management.
-

4. Protocol Workflow

1. **Payload Generation:** The Issuer submits the verified candidate details and cryptographically signs the identity payload, establishing the root of trust.
2. **Secure Synchronization:** The payload is securely transmitted to the Candidate's Identity Vault, transferring data custody to the user.
3. **Policy Configuration:** The Enterprise Verifier defines a cryptographic constraint utilizing specific logical operators ($>$, $=$, $<$, $\leq$, $\geq$, $\neq$) against targeted attributes.
4. **Zero-Knowledge Computation:** The Candidate reviews the requested policy and initializes proof generation. The Identity Vault redacts all PII, evaluates the hidden witness data against the verifier's constraints, and constructs a valid zk-SNARK. If constraints are unmet, the circuit aborts execution.
5. **Gasless Execution:** The protocol routes the transaction through an ERC-4337 Paymaster, subsidizing transaction fees for the prover.
6. **Verification & Auditing:** The Enterprise Verifier receives the proof, validates the mathematical constraints via the pairing matrix, and outputs a boolean assertion. The transaction hash, timestamp,

and evaluated policy are permanently recorded on the blockchain ledger.

5. Local Installation & Environment Setup

5.1. Prerequisites

Ensure the following runtimes and tools are installed on your host system:

- **Node.js:** v18.17.0 or higher
- **npm:** v9.0.0 or higher (or **pnpm** / **yarn**)
- **Git:** Latest stable version

5.2. Repository Initialization

Clone the repository to your local directory:

```
git clone [https://github.com/Balavardhan01/ZKP-Credential-Verifier_Avengers_Avirbhav.git](https://github.com/Balavardhan01/ZKP-Credential-Verifier_Avengers_Avirbhav.git)
cd ZKP-Credential-Verifier_Avengers_Avirbhav
```

5.3. Smart Contract Layer (Hardhat)

Initialize and compile the verification contract environment:

```
cd contracts
npm install
npx hardhat compile
```

(Optional) Start a local Hardhat node in a dedicated terminal if deploying locally:

```
npx hardhat node
```

5.4. Frontend Client Layer (Next.js)

In a separate terminal session, install frontend dependencies and start the local development server:

```
cd ../frontend
npm install
npm run dev
```

The application client will initialize at:

```
http://localhost:3000
```

6. User Acceptance Testing (UAT) / Execution Guide

To validate the system architecture, open three separate browser tabs side-by-side within the same browser profile. If using the live demo, replace `localhost:3000` with the deployed URLs.

- **Issuer Session:** `http://localhost:3000/issuer`
- **Student/Prover Session:** `http://localhost:3000/student`
- **Verifier Session:** `http://localhost:3000/verifier`

6.1. Initialization (Root of Trust)

1. Access the **Issuer Session**.
2. Authenticate using institutional credentials.
3. Within the "Manual Issuance" module, input candidate parameters (e.g., Name: Jane Doe, GPA: 3.8, College: Engineering, Degree: B.S. CS, Year Passed: 2024).
4. Execute **"Sign & Generate Payload"**.

6.2. Data Custody Verification

1. Transition to the **Student Session**.
2. Authenticate to decrypt the Identity Vault.
3. Confirm the Vault displays the corresponding Digital ID card with the values minted in Section 6.1.
4. Verify the **0.00 MATIC (ERC-4337 Sponsored)** indicator, confirming the active gas abstraction layer.

6.3. Policy Configuration & Execution

1. Transition to the **Verifier Session**.
2. Define a passing constraint (e.g., Attribute: `Cumulative GPA`, Operator: `>=`, Threshold: `3.0`).
3. Return to the **Student Session** and execute **"Obfuscate Data & Generate ZKP"**.
4. Observe the localized PII obfuscation (`[ ZK-OBFUSCATED ]`) and the live terminal computation of R1CS polynomial constraints.

6.4. Audit & Verification

1. Transition back to the **Verifier Session**.
2. Observe the Live SIEM matrix processing the elliptic curve pairing checks.
3. Upon validation, confirm the system outputs **"Proof Validated"** (**Circuit Output: TRUE**).
4. Execute the **"Receipt"** export to download the W3C-standard Verifiable Presentation JSON.
5. Review the Immutable On-Chain Ledger to confirm the transaction hash and timestamp entry.

6.5. Constraint Rejection (Anti-Fraud Validation)

1. In the **Verifier Session**, select **"Reset / New Request"**.
2. Configure an unsatisfiable constraint (e.g., Attribute: **Year Passed**, Operator: **==**, Threshold: **2028**).
3. In the **Student Session**, initiate proof generation.
4. Observe the local circuit terminal halt execution (**CONSTRAINT_ERROR**).
5. Confirm the **Verifier Session** registers the event as **"Proof Rejected"** (**Circuit Output: FALSE**) and appends a rejection entry to the audit ledger.

7. Future Technical Roadmap

- **On-Chain Solidity Verification:** Transitioning from localized verification simulation to a fully deployed **Verifier.sol** smart contract on the Polygon Amoy testnet, utilizing **ethers.js** for decentralized consensus.
- **Zero-Knowledge Revocation via Merkle Trees:** Integration of a privacy-preserving revocation registry. Provers will utilize Merkle Tree exclusion proofs to cryptographically guarantee valid credential status without exposing leaf indices or identity vectors.
- **Physical-to-Digital Bridge:** Porting the Identity Vault to a React Native mobile application leveraging the device's Secure Enclave, enabling tap-to-verify NFC interactions for localized physical access control.